SOFTWARE ENGINEERING PROJECT REPORT

CineVerse

Movie Ticket Booking System

Requirements • Design • Implementation • Testing

PREPARED BY

[Add team member names and student IDs]

SUBMITTED TO

[Add faculty name, course, department and college]

2026

Project at a glance

CineVerse is a full-stack classroom prototype for discovering movies and completing a simulated cinema booking. It combines a static JavaScript frontend with a Spring Boot REST backend and JPA persistence. Customers can choose a city, movie, show, seats and concession items; the backend validates booking requests and saves a booking summary. An authorized administrator can review booking statistics and screening prices.

Important scope note	Cinema listings and payment are demo/simulated in the supplied screenshots. The displayed QR reference is illustrative, not scannable; no real payment is processed.

Abstract

This report documents the analysis, design, implementation and verification of CineVerse, a movie ticket booking web application. The system demonstrates a complete customer flow from account access through movie and show selection, seat and food selection, billing and ticket display. Its admin module provides report summaries, reservation search and screening-price management. The active development configuration uses an H2 in-memory database, so demo data is recreated when the application restarts. The report includes the system architecture, workflow, simplified domain model, implementation evidence, test coverage and project limitations.

Contents

- 1. Introduction and problem definition
- 2. Objectives, scope and stakeholders
- 3. Requirements specification
- 4. Architecture and UML-style design views
- 5. Data design and business rules
- 6. Implementation and user-interface evidence
- 7. Verification and test coverage
- 8. Challenges, limitations and future work
- 9. Conclusion and references

1. Introduction

Online movie booking involves more than selecting a title: users need a clear route through location, screening, seat availability and order totals. CineVerse models that journey in a browser-based application. The backend owns important decisions such as account roles, booking validation, seat reservation and totals, while the frontend presents the workflow and current selection.

1.1 Problem statement

Users need a straightforward way to find a movie and screening for a selected city, understand seat and food charges before confirming, and retrieve their booking afterwards. Administrators need protected access to booking summaries and screening prices. The project addresses these requirements with a guided customer workflow and a role-protected reporting module. Demo theatre inventory is explicitly labelled where used.

1.2 Objectives

- Provide customer registration, login and account-aware booking access.
- Support city, movie, cinema and show selection.
- Display seat availability and apply premium-seat pricing rules.
- Include concession items in the booking total.
- Persist booking details and provide a confirmation/ticket view.
- Restrict administration functions to an authenticated ADMIN role and configured Admin ID.
- Demonstrate maintainable separation between web controllers, services, repositories and entities.

1.3 Scope and constraints

- In scope: customer accounts, movie discovery/catalog, booking shows, seats, food orders, booking aftercare, admin reporting and screening-price management.
- Demo constraint: the default datasource is H2 in-memory and uses create-drop schema behavior; records do not persist across application restarts.
- Demo constraint: payment status is simulated; there is no payment processor or real financial transaction.
- Demo constraint: fallback cinemas and showtimes are labelled as demo data and must not be presented as real theatre availability.
- The current application configuration selects H2; a PostgreSQL driver is also declared in Maven, but a PostgreSQL datasource is not active in application.properties.

1.4 Stakeholders

Stakeholder	Goal / responsibility
Customer	Find a film and show, reserve seats, add snacks, review and retrieve a booking.
Administrator	Review booking reports and update screening prices using protected access.
Project team	Analyze requirements, implement, test, document and maintain the prototype.
Faculty / evaluator	Assess requirements, design decisions, implementation evidence and testing.

2. Requirements specification

2.1 Functional requirements

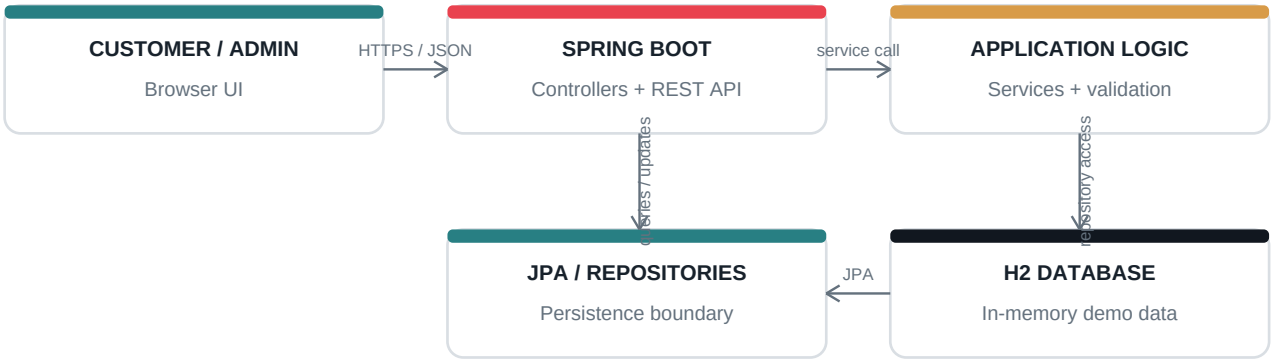
ID	Requirement
FR-01	A visitor can register a customer account and sign in.
FR-02	A customer can select a city and browse movies and movie details.
FR-03	A customer can request available cinemas and showtimes for a movie, city and date.
FR-04	A customer can inspect seat availability and select seats for a show.
FR-05	The booking service calculates seat and food subtotals and the total server-side.
FR-06	A customer can confirm a demo booking and retrieve its ticket/booking summary.
FR-07	A customer can view their own bookings and submit eligible aftercare requests.
FR-08	An admin must pass admin ID, email, password and ADMIN role checks.
FR-09	An admin can view filtered dashboard reports and reservation data.
FR-10	An admin can update a screening's ticket price; changes apply to future bookings.

2.2 Non-functional requirements

- Security: password hashes are stored; admin functions require an ADMIN role; state-changing requests use CSRF verification.
- Integrity: the backend validates booking data and prevents duplicate reservations for the same show and seat.
- Usability: the interface presents the booking as sequential, visible steps and explains simulated inventory/payment.
- Maintainability: controllers, services, repositories, DTOs and JPA entities are separated by responsibility.
- Portability for development: Maven and an embedded H2 database avoid requiring a separately installed database server.

3. Architecture and design

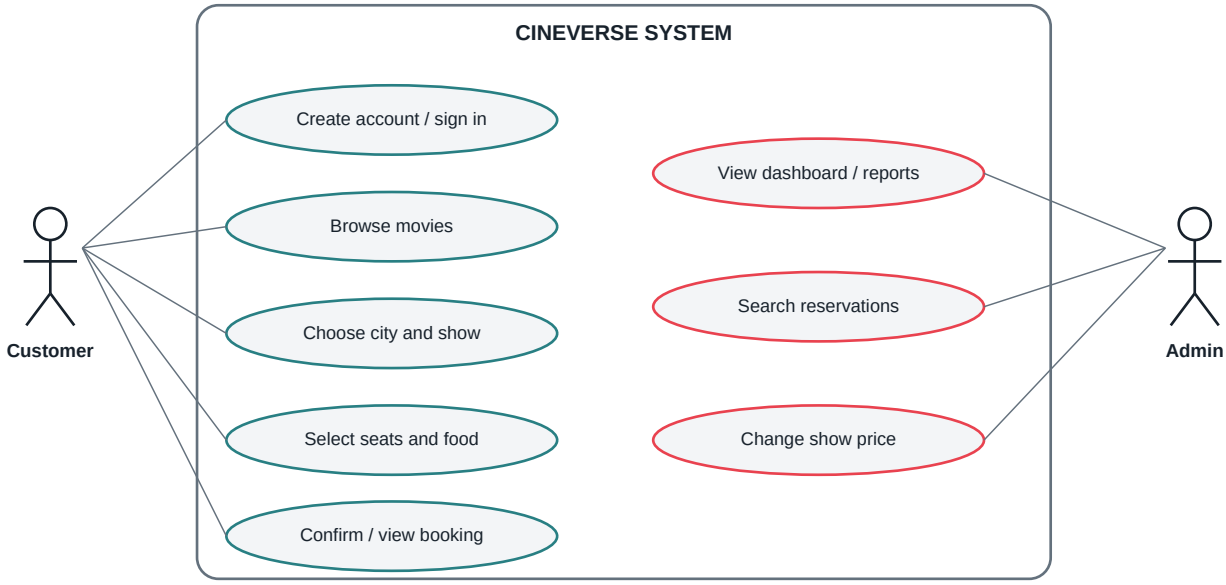
The application follows a client-server, layered design. The browser renders the static frontend and calls REST endpoints. Controllers validate/route HTTP requests, services apply booking and access rules, repositories use Spring Data JPA, and entity classes map the domain to relational tables. The current development profile stores data in H2 in-memory.



Logical view of the current application; external payment processing is not implemented.

Figure 1. Logical application architecture based on the project structure and active datasource configuration.

3.1 UML use-case diagram



Admin use cases require an ADMIN account; sign-in also checks the configured Admin ID.

UML U1. Customer and administrator use cases within the CineVerse system boundary.

3.2 Customer booking workflow

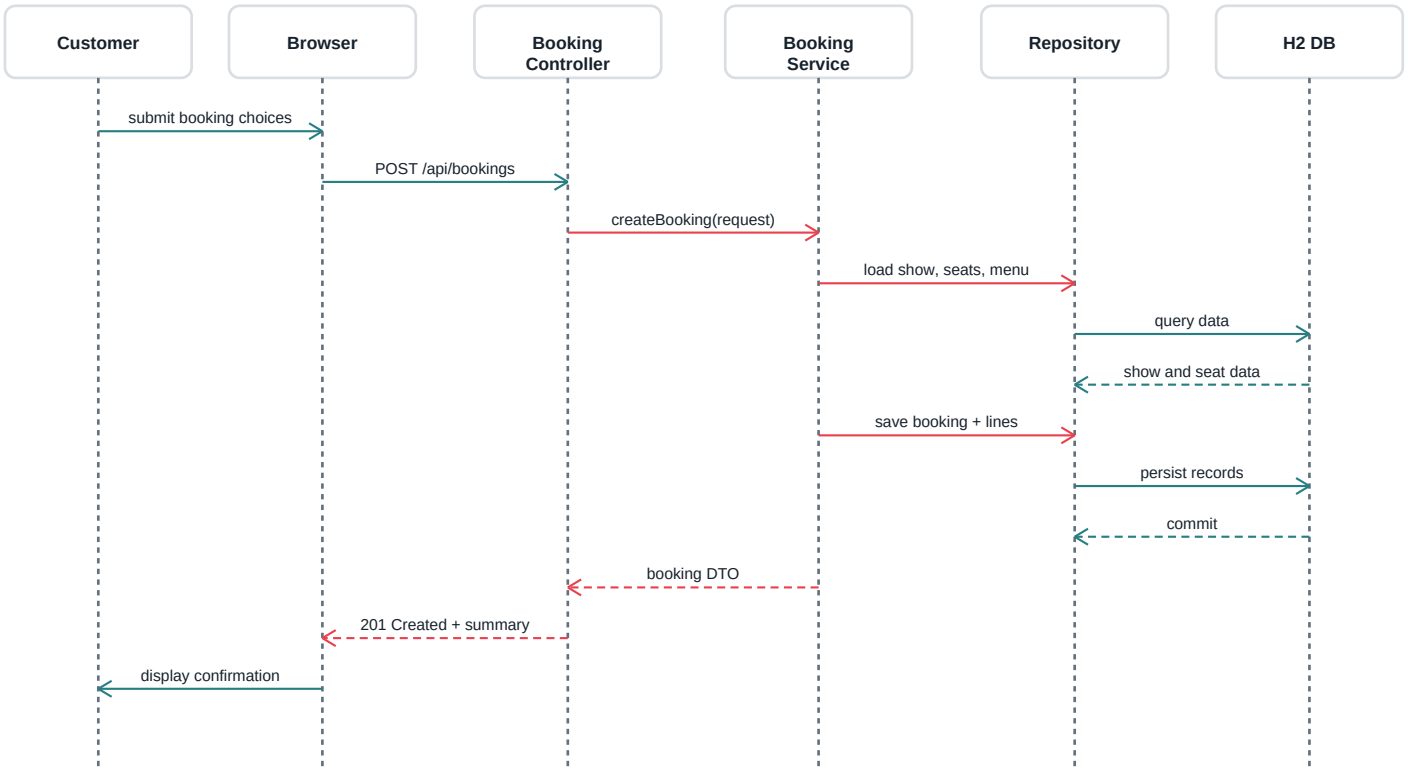


Admin path: separate admin sign-in → dashboard → report filters / booking list / show pricing.

Figure 2. Customer booking workflow. The admin path is separate and guarded by role and Admin ID checks.

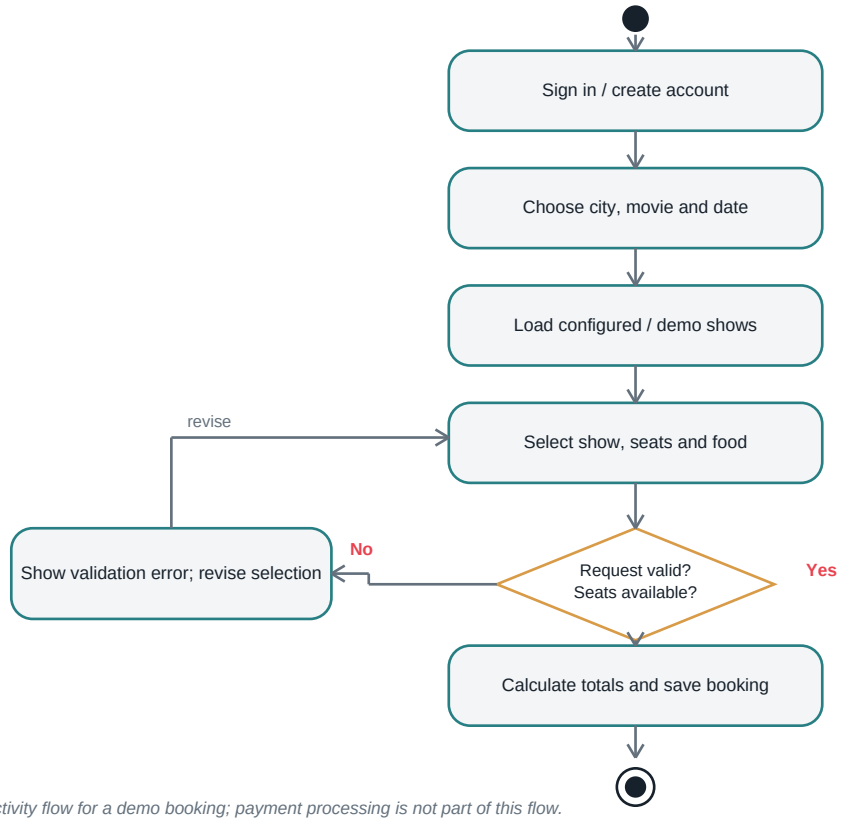
3.3 Booking sequence

Step	Interaction	Responsibility
1	Customer selects a show, seats, food and payment method.	Frontend collects the requested choices.
2	Frontend sends a booking request to the REST API.	Controller validates request shape and session.
3	Booking service loads show/seats and checks availability.	Service enforces booking rules and computes authoritative prices.
4	JPA persists booking, seats and food lines.	Repositories write to the configured H2 datasource.
5	API returns booking DTO; UI renders confirmation.	Frontend displays reference, subtotals and simulated payment status.



UML U3. Sequence diagram for booking creation, server-side validation, persistence and response. Payment is simulated.

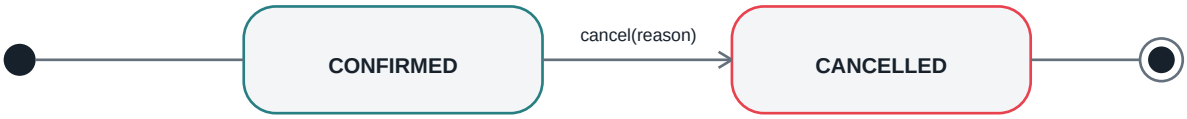
3.4 Booking activity diagram



Activity flow for a demo booking; payment processing is not part of this flow.

UML U4. Activity diagram with the validation/seat-availability decision and correction path.

3.5 Booking state diagram



A successful booking is created as CONFIRMED; cancellation changes it to CANCELLED and stores reason/time.

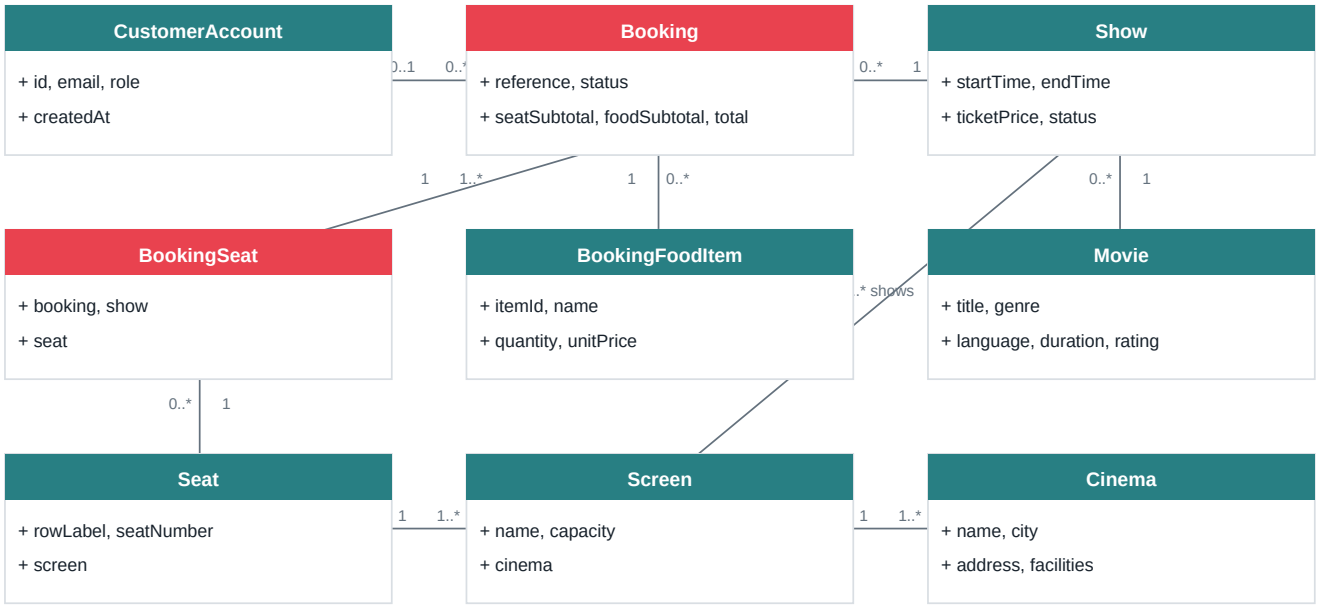
Aftercare is a separate field: requesting it sets aftercareStatus = PENDING; it is not a BookingStatus state.

UML U5. Booking lifecycle states implemented by BookingStatus.

4. Data design

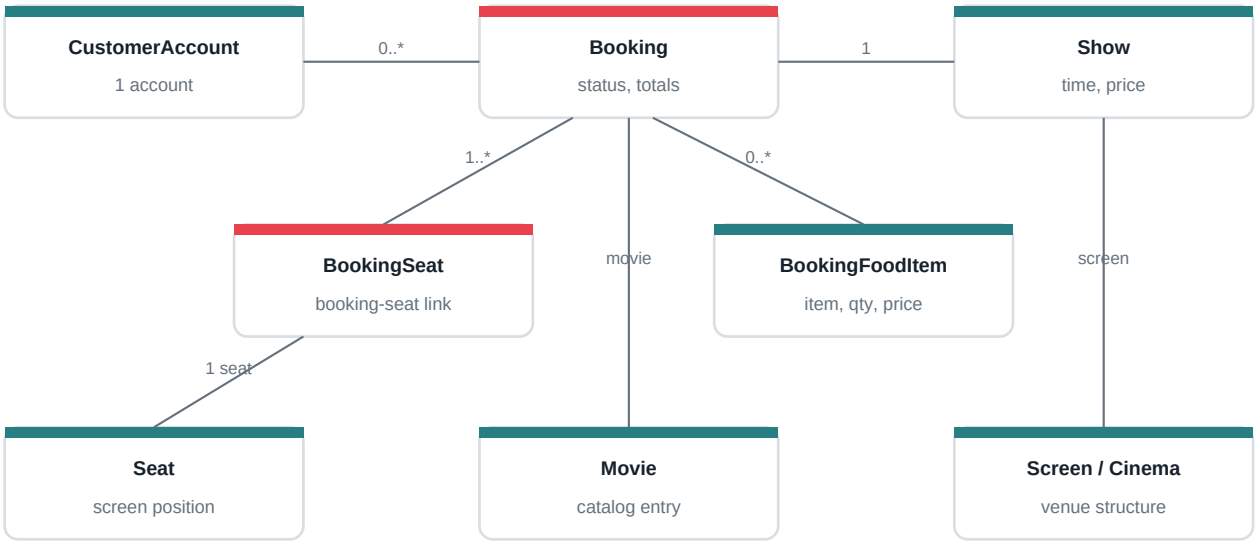
JPA entities represent customers, movies, cinemas, screens, seats, shows and bookings. BookingSeat links a booking to a seat for a specific show; BookingFoodItem records the purchased item name, quantity and unit price. Booking stores seat subtotal, food subtotal, total, status and optional aftercare details. The diagram is intentionally simplified and omits implementation fields that are not needed to understand the relationships.

4.1 UML class diagram



Simplified UML domain class view; DTOs, repositories and service classes are omitted for readability.

UML U2. Core domain classes, selected attributes and multiplicities based on the JPA entities.



Simplified domain model. Show references Movie and Screen; Screen belongs to a Cinema.

Figure 4. Simplified domain/entity relationship view. Cinema owns screens; each show references a movie and a screen.

4.2 Key business rules

- A booking belongs to one show and may contain several booked seats and food lines.
- A database uniqueness constraint prevents the same seat from being reserved twice for one show.
- The booking service calculates totals using server-side show pricing and the allowed food menu; the browser is not authoritative for price.
- Premium seats are identified by the configured front-row rule; standard seats use the base show price.
- Cancelling a booking changes its status and records a reason/time; aftercare requests have their own type and status.
- The Admin dashboard is restricted to the ADMIN account role; the configured Admin ID is also checked during admin sign-in.

4.3 Main data entities

Entity	Purpose
CustomerAccount	Customer identity, encoded password and account role.
Movie	Movie catalog and details shown during discovery/booking.
Cinema / Screen / Seat	Venue hierarchy and physical seating layout.
Show	Scheduled movie on a screen with time, status and ticket price.
Booking	Reservation reference, customer/show, status, subtotals and total.
BookingSeat	Join entity connecting a booking, show and reserved seat.
BookingFoodItem	Food order line with item ID, name, quantity and unit price.

5. Technology and implementation

Layer	Technology / implementation
Frontend	Static HTML, CSS and JavaScript served by Spring Boot.
Backend	Java 17, Spring Boot 3.5.7, REST controllers and service layer.
Persistence	Spring Data JPA / Hibernate with H2 in-memory datasource in application.properties.
Build	Maven; Spring Boot Maven plugin.
Verification	JUnit 5, Spring Boot Test and MockMvc integration tests.
Version control	Git repository hosted on GitHub.

5.1 Object-oriented and design concepts

- Encapsulation: entity state is private and accessed or changed through methods, keeping persistence objects responsible for their data.
- Abstraction: controllers work with DTOs and services rather than exposing JPA entities directly to the browser.
- Association: the domain models relationships such as Cinema–Screen, Show–Movie, Show–Screen, Booking–Show, Booking–BookingSeat and Booking–BookingFoodItem.
- Single responsibility / layered design: controllers handle HTTP, services enforce business rules, repositories handle persistence, and DTOs define API payloads.
- Enums: account role, booking status, show status and aftercare states constrain domain values to explicit options.
- Validation and defensive checks: request validation, session/role checks and server-side price calculation protect key operations.

5.2 Booking total calculation

Total	Booking total = server-calculated seat subtotal + server-calculated food subtotal. The checkout screen shows payment choices, but completing the booking records a simulated result rather than charging a payment method.

Project planning and management

Plan status	The schedule and role split below are a proposed planning template, not a claim about the team's actual historical dates or individual contributions. Replace the role placeholders with your group's real details.

Phase	Suggested duration	Work and deliverables
1. Requirements	Week 1	Confirm problem, users, scope, functional/non-functional requirements and acceptance criteria.
2. Analysis and design	Week 2	Review use cases, UML models, architecture, entity relationships and UI flow.
3. Implementation	Weeks 3-4	Implement frontend, Spring Boot APIs, booking rules, persistence, admin functions and integration.
4. Verification	Week 5	Run unit/integration/system tests, record defects, fix issues and repeat regression tests.
5. Documentation and demo	Week 5	Prepare screenshots, report, presentation, run instructions and demonstration checklist.

Suggested responsibilities for a group of 2-3

Role placeholder	Suggested ownership
Member A: [name / ID]	Requirements, UML, documentation and coordination.
Member B: [name / ID]	Backend entities/services/APIs, persistence and backend tests.
Member C: [name / ID, if applicable]	Frontend workflow, UI integration, screenshots and demo preparation.
Shared	Design reviews, integration testing, defect triage and final presentation.

6. User-interface evidence

The following figures are captures from the supplied running application. They document the customer journey and administration interface. City listings and sample cinema inventory shown in the captures are demo data. Personal signup/profile screenshots were intentionally omitted from this report to avoid publishing the user's email address.

6.1 Home and featured movie

The home page introduces the movie discovery experience and provides navigation to movies, bookings and account actions.

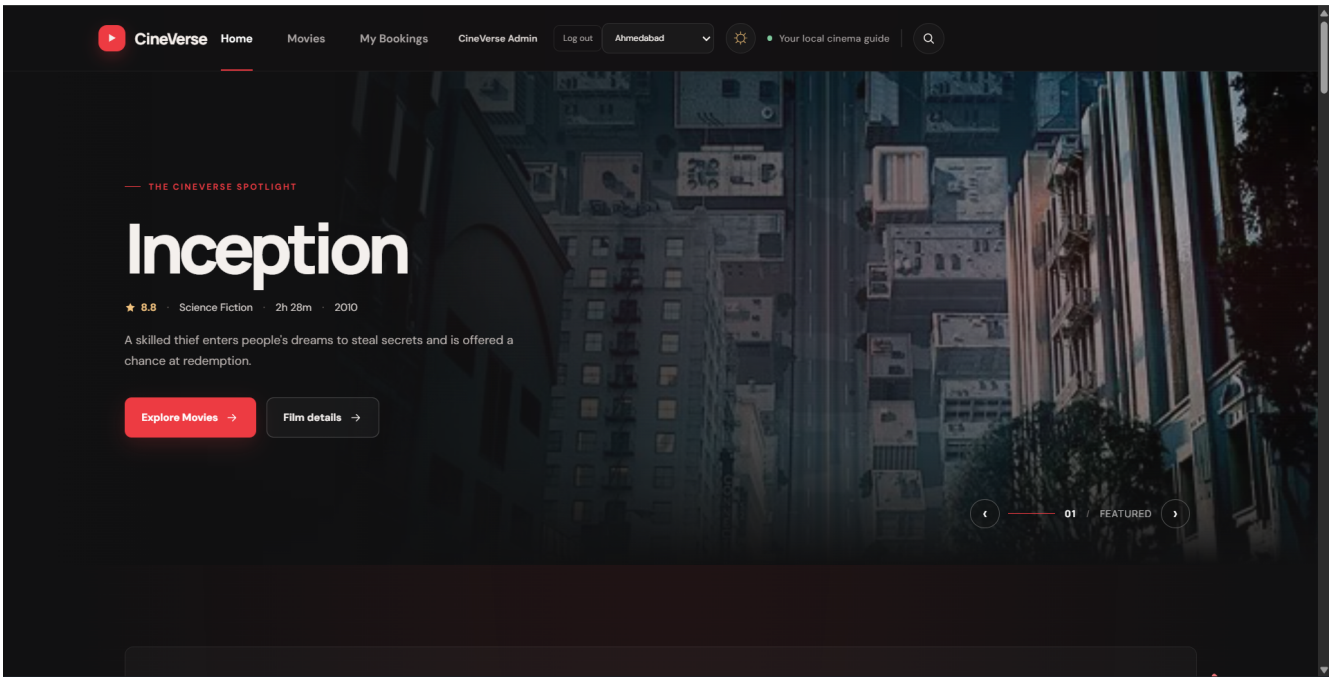


Figure 5. CineVerse homepage with featured film, navigation and city selector.

6.2 Movie details

The movie page presents title details and gives the user a direct route into the booking flow.

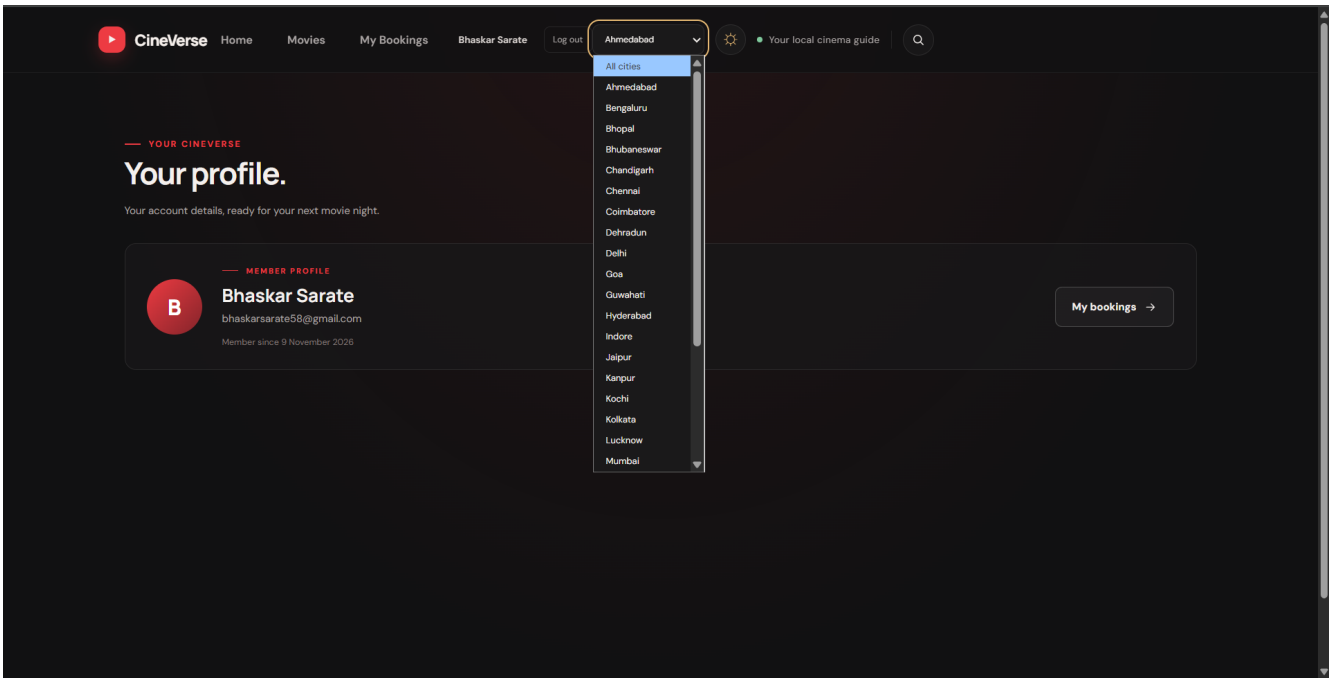


Figure 6. Movie details page with film information and booking entry point.

6.3 City and show-date selection

The booking flow keeps the selected city visible while the customer chooses an available show date.

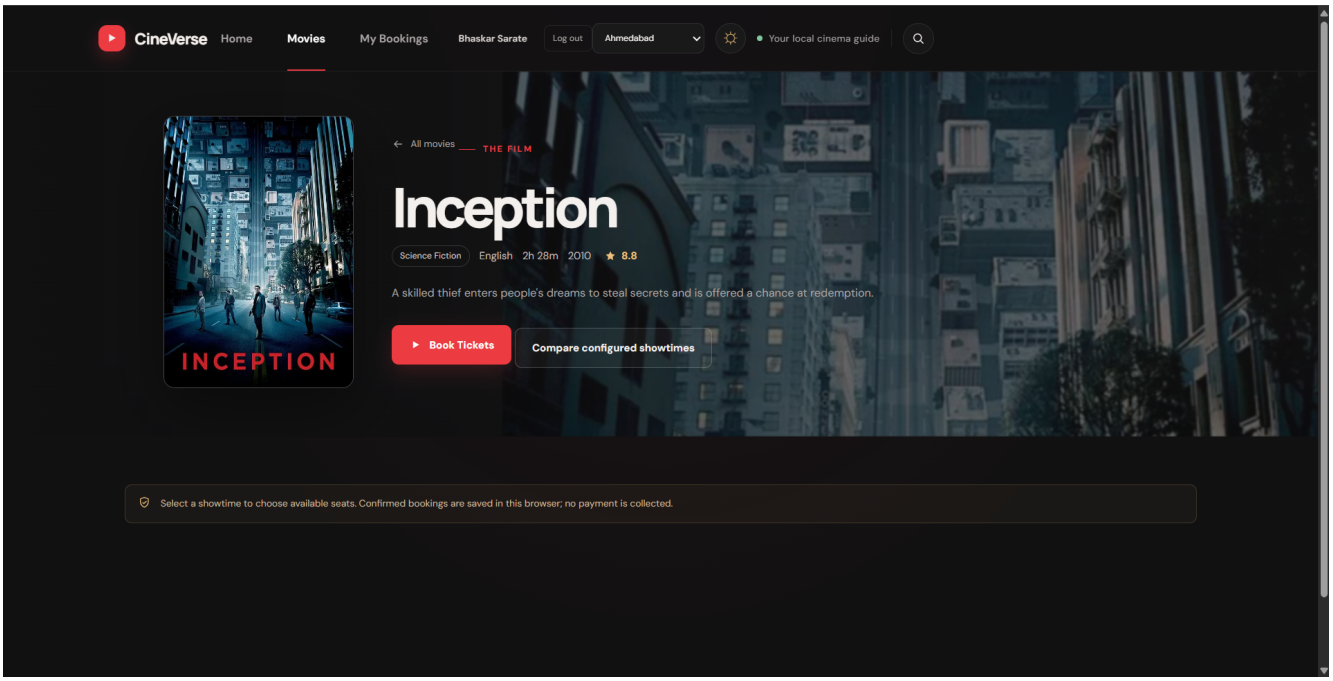


Figure 7. Show-date selection with the active city displayed.

6.4 Cinema and showtime choices

This capture shows the cinema grouping and showtime cards. These locations are labelled DEMO and represent simulated inventory, not real theatres.

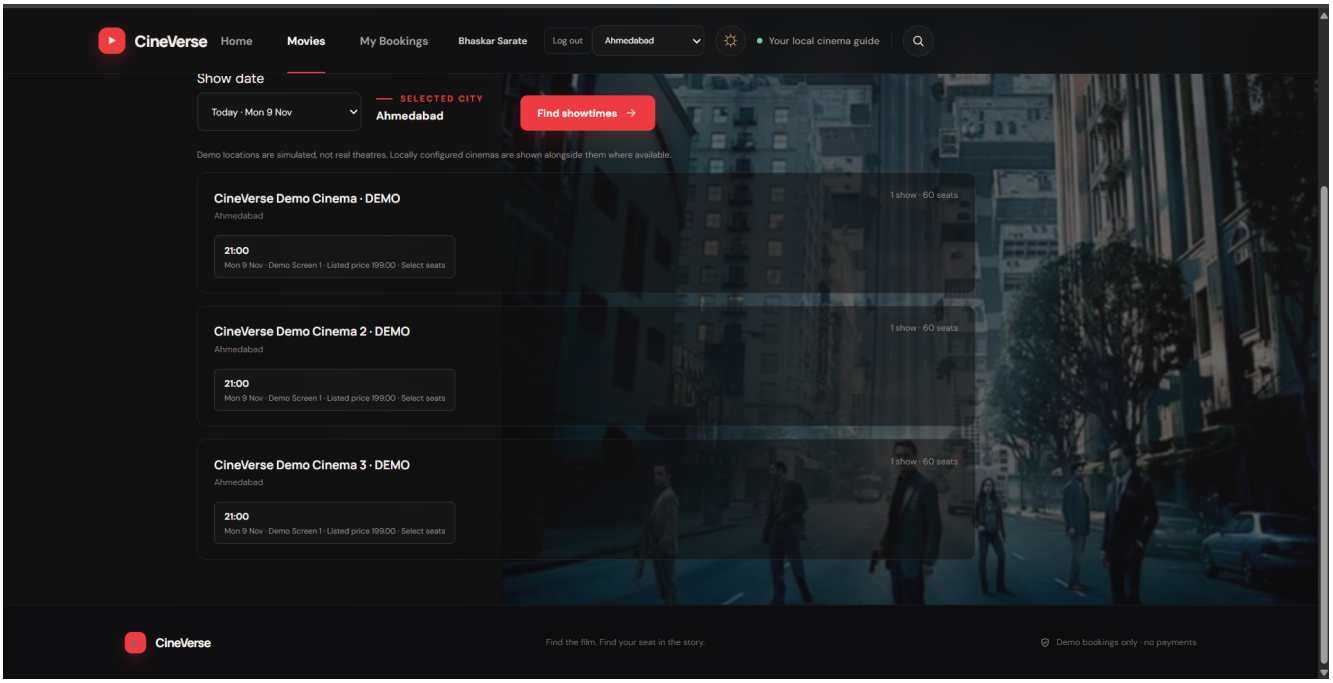


Figure 8. Example city results with three clearly marked demo cinema options.

6.5 Seat and food selection

The selection view distinguishes front-row premium seats from standard rows and includes concession quantities in the displayed amount.

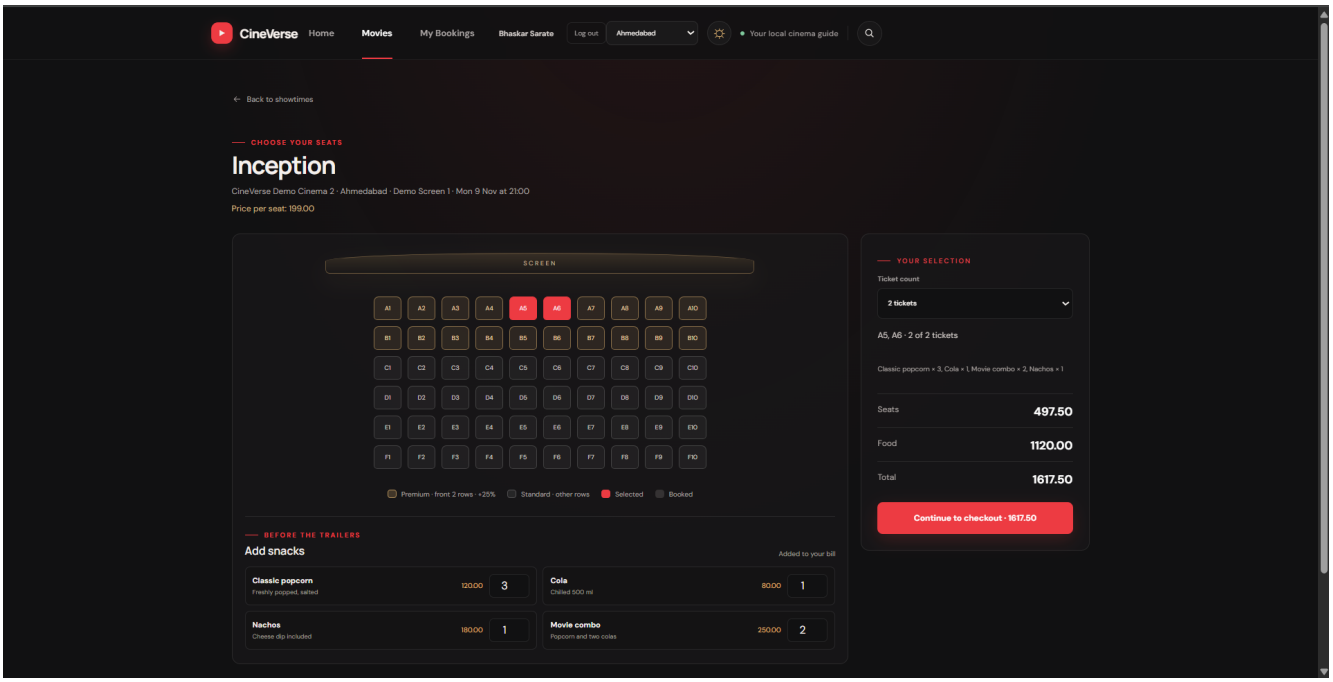


Figure 9. Seat layout with premium/standard legend, snack quantities and live total.

6.6 Checkout review

The review page itemizes the booking before confirmation. Payment methods shown are demonstration choices only.

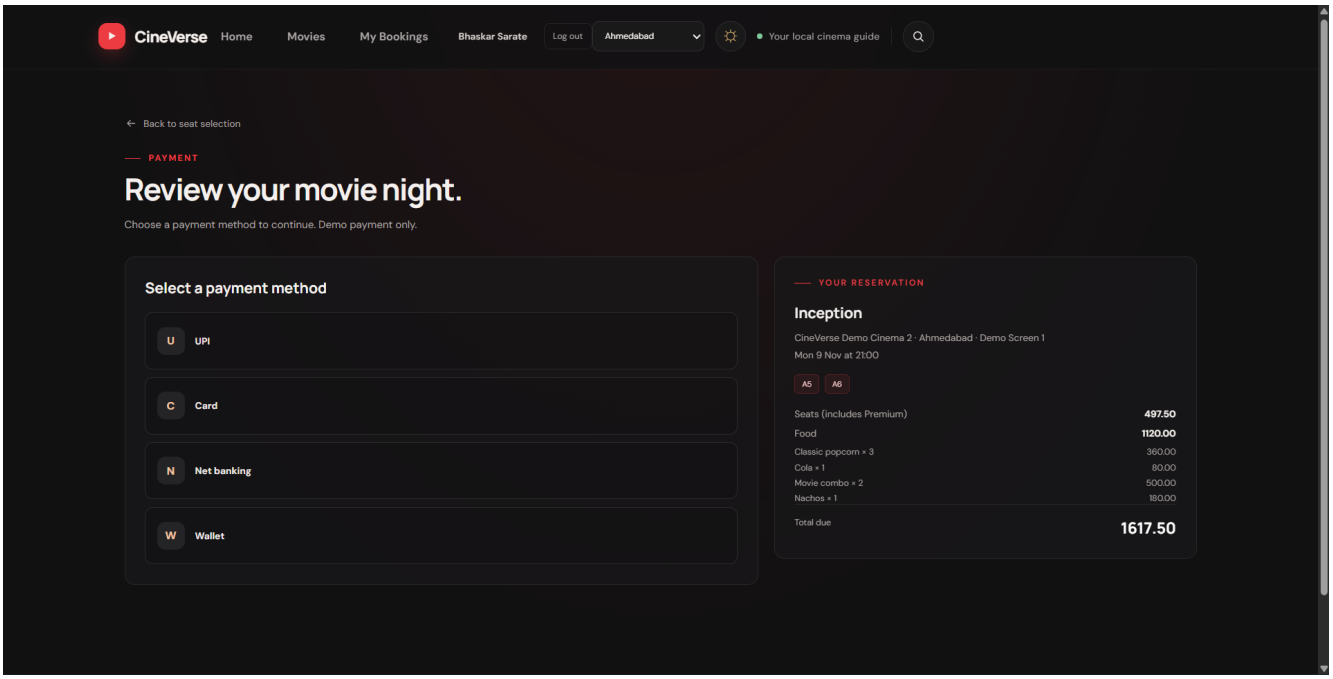


Figure 10. Simulated checkout showing seat subtotal, food subtotal and total due.

6.7 Booking confirmation

The booking detail view presents the saved demo reservation and its server-returned totals. The reference/QR is explicitly illustrative and not scannable.

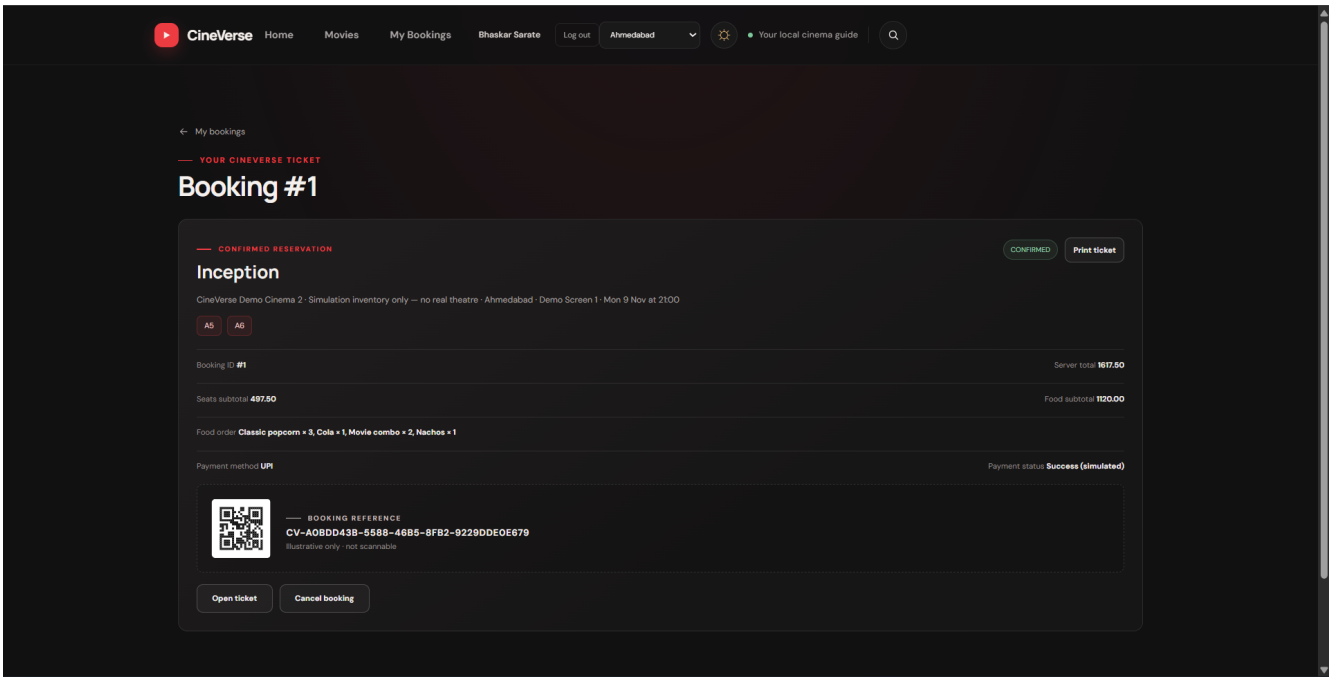


Figure 11. Confirmed demo booking with seats, subtotals and illustrative reference.

6.8 Admin dashboard overview

The administrator view summarizes catalog, cinema, show, account, booking and demo-revenue measures. Values are from the captured demo dataset, not production statistics.

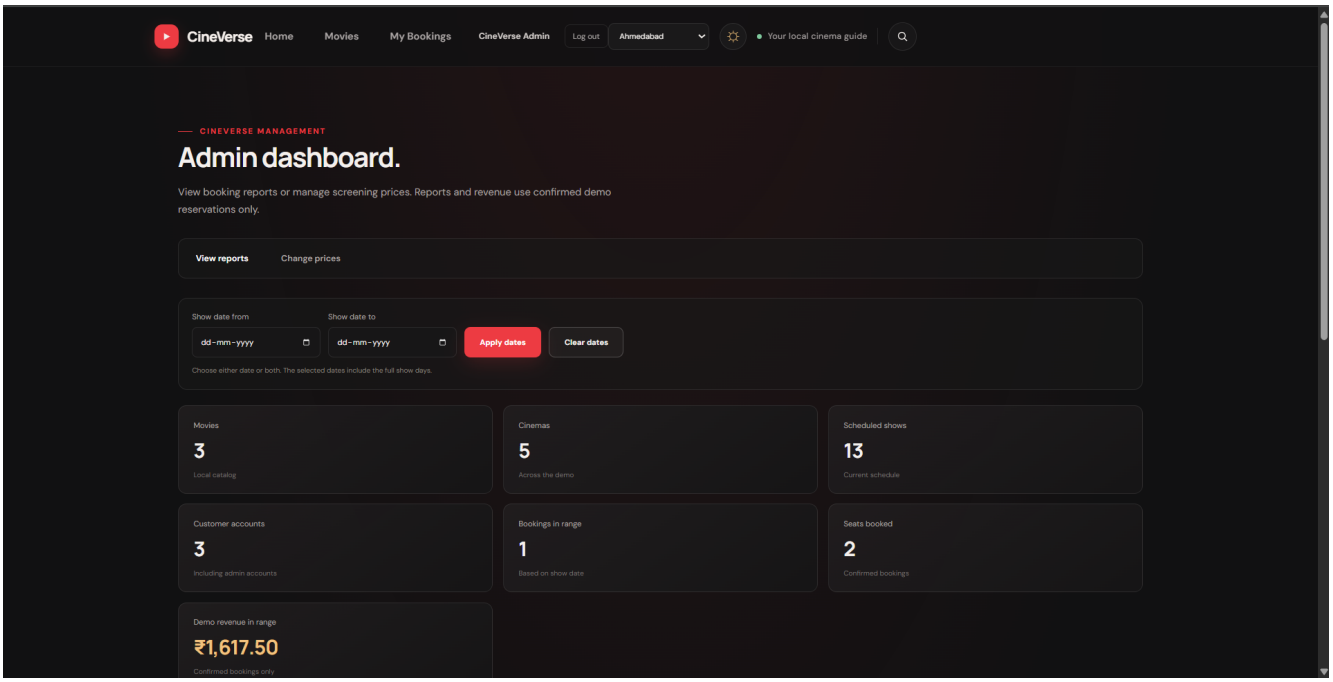


Figure 12. Admin dashboard summary metrics and date filters.

6.9 Reports and reservations

The report view includes grouped booking metrics, payment/cancellation summaries, reservation search and CSV export controls.

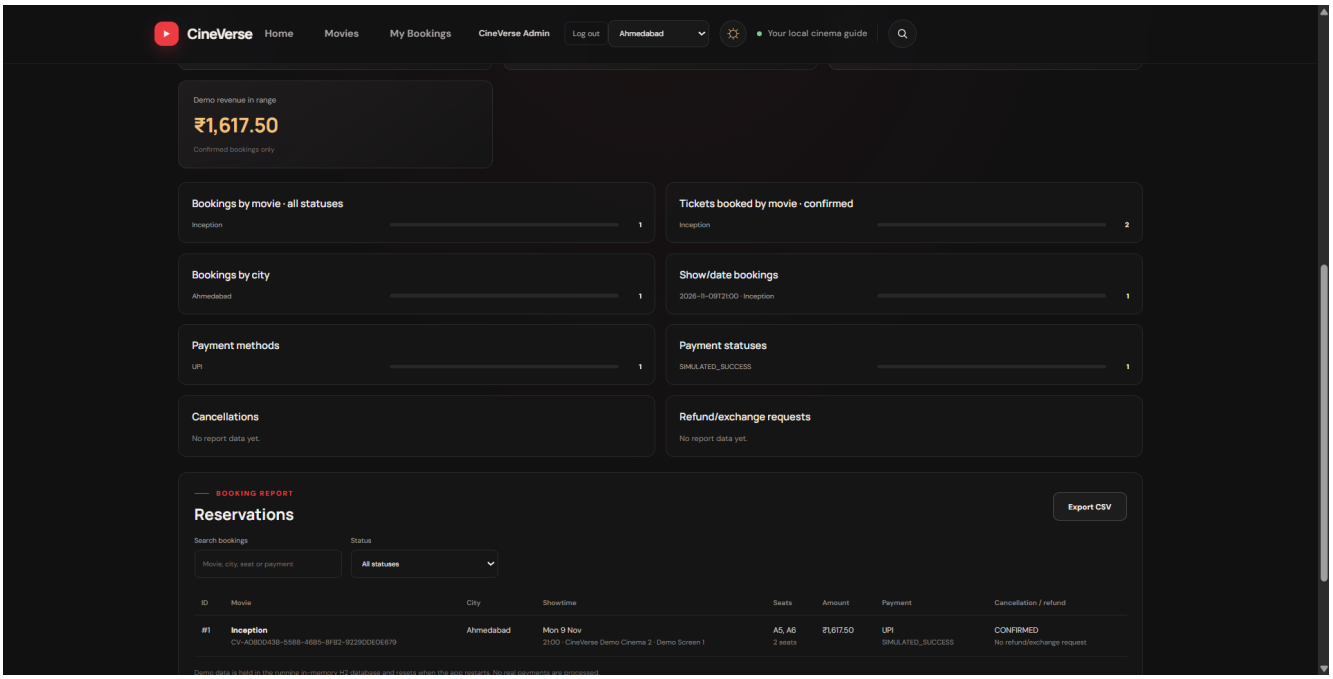


Figure 13. Admin booking breakdowns and reservation report table.

7. Verification and testing

The repository includes automated Spring Boot/MockMvc tests for core booking and administrator behavior. The following results are from an actual `mvn test` run on 10 Nov 2026.

Area	Tested behavior in source	Evidence file
Booking	Create a confirmed booking, calculate totals, verify seat availability and retrieve the booking.	BookingControllerTest
Pricing / food	Check premium and standard seat totals, food menu and persisted food line totals.	BookingControllerTest
Show fallback	Use configured shows first and return at least three explicitly flagged demo choices when needed.	BookingControllerTest
Admin authentication	Reject customer role, wrong admin ID and admin login through the customer endpoint; accept configured admin credentials.	AdminControllerTest
Authorization / CSRF	Reject anonymous/customer report access and reject state-changing requests without a valid CSRF token.	AdminControllerTest
Admin reporting	Filter dashboard data by show date and reject invalid date ranges.	AdminControllerTest

Actual test run Command: `mvn test` (10 Nov 2026). Result: 52 tests, 49 passed, 3 failed, 0 errors, 0 skipped; Maven reported BUILD FAILURE. These are observed assertion mismatches; root causes have not yet been diagnosed.

Failing test	Observed assertion
AdminControllerTest.adminReportIsRoleProtectedAndFiltersBookingsByShowDate	Expected totalRevenue 350.0; received 437.5.
AuthControllerTest.customerBookingReportAndBookingLookupAreRestrictedToOwner	Expected HTTP 201; received HTTP 400.
CinemaShowControllerTest.developmentDataLoaderIsIdempotent	Expected 3 records; found 10.

All 15 BookingControllerTest cases passed in this run. Other passing suite counts: DiscoveryControllerTest 3/3, HealthControllerTest 1/1 and MovieControllerTest 12/12. AdminControllerTest and AuthControllerTest each had 3 passed and 1 failed; CinemaShowControllerTest had 12 passed and 1 failed.

7.1 Sample manual acceptance checks

Check	Expected observation
Customer flow	City, movie, date, cinema and show are carried forward through seat selection.
Seat selection	Selected seats appear in the summary; occupied seats are unavailable.
Food calculation	Changing food quantity changes the food subtotal and total.
Booking confirmation	Confirmation displays show, seats, food, subtotals and demo payment status.
Admin boundary	Customer accounts cannot access admin report or price management endpoints.

8. Challenges, limitations and future work

8.1 Challenges and design responses

Challenge	Response in the project
Preserving user context across booking/auth steps	Carry the selected city and return route through the frontend flow.
Avoiding client-controlled totals	Calculate seat/food totals in the backend and return authoritative booking values.
Preventing duplicate seat sales	Validate availability and enforce a database uniqueness constraint per show/seat.
Providing theatre choices with sparse local inventory	Use clearly labelled demo cinema/show records where suitable real configured data is absent.
Protecting admin operations	Require the ADMIN role, configured Admin ID and CSRF verification for protected actions.

8.2 Limitations

- H2 is in-memory under the current configuration; data resets after application restart.
- The payment flow is simulated; no card, UPI or wallet transaction occurs.
- Some cinema/show data is demonstration inventory, not live theatre availability.
- The booking reference QR in the current UI is explicitly illustrative and not a scannable admission credential.
- The report screenshots show one demo dataset and should not be interpreted as live operational analytics.

8.3 Maintenance plan

When	Maintenance activity
For every code change	Run focused tests and `mvn test`; review validation, authorization and booking-total behavior; update related documentation.
Each release	Build from a clean checkout, verify environment configuration, review logs and confirm demo/production labels.
Monthly or before a submission	Review Spring Boot/Maven dependency updates, security advisories, open defects and UI compatibility.
Before production use	Replace in-memory H2 with persistent storage, add migrations/backups, protect secrets, configure monitoring and integrate real payment processing.
After an incident	Reproduce the issue, preserve diagnostic logs without secrets, add a regression test, fix and re-verify.

Assign a maintainer for each release. Admin credentials and API keys must remain in environment/configuration secrets and must not be added to source control.

8.4 Future enhancements

- Move to a persistent production database and define migration/backup procedures.
- Integrate a compliant payment provider with secure transaction status handling.
- Issue verifiable QR tickets and add entry validation at the venue.
- Connect to live cinema/showtime inventory and notify users of booking changes.
- Add email/SMS notifications, ratings, accessibility options and richer analytics.

9. Conclusion

CineVerse demonstrates a complete movie reservation workflow and applies software-engineering practices across requirements, modular design, persistence, access control, pricing and testing. The project supports city and movie discovery, show selection, seat and food choices, server-calculated booking totals, ticket summaries and admin reporting. Its current H2 and payment configuration is intentionally suitable for a classroom demo, not a live ticketing service. The included test sources and visual evidence provide a basis for evaluating the implementation, while persistent storage, live inventory and real payment processing remain clear future extensions.

References

- Spring Boot Reference Documentation, https://docs.spring.io/spring-boot/
- Spring Data JPA Reference Documentation, https://docs.spring.io/spring-data/jpa/
- H2 Database Documentation, https://www.h2database.com/
- Java Platform Documentation, https://docs.oracle.com/en/java/
- CineVerse source code, project README and automated test sources.

Appendix A. Run the project

From the project root, run `mvn spring-boot:run`. The default port is 8080, configurable with the PORT environment variable. The development datasource is embedded H2, so no separate database installation is required. For a submission demonstration, use only demo accounts and sample data; do not include passwords, Admin IDs or private environment values in the report.

Before submission	Fill in the cover-page team/faculty/college fields, replace proposed planning placeholders with your group's real details, and review/fix the three failing tests before claiming the suite passes. Review screenshots for private information before sharing the PDF.